Guia de Uso do Kafka — Transporte de Eventos (B2)

Este documento descreve como qualquer componente do sistema deve se conectar ao serviço Kafka, qual biblioteca cliente usar conforme a linguagem do componente, a nomenclatura de tópicos adotada e um passo a passo mínimo de publicação e consumo. Serve como referência rápida para todos os squads que publicam ou consomem eventos pelo barramento.

> **Importante:** neste momento o serviço Kafka está rodando na nuvem (Google Cloud) apenas para fins de teste. Na versão final do sistema, ele estará rodando localmente. Por isso, o endereço indicado neste documento é temporário e os componentes devem deixar o endereço do broker configurável (e não fixo no código), para que a troca seja feita sem alterar o restante da aplicação.

1. Onde está o serviço

Endereço (host:porta): 34.66.235.91:9093

Ambiente: Google Cloud — instância compartilhada por todos os squads do projeto.

Este é o endereço de bootstrap do cluster Kafka. Todo cliente, em qualquer linguagem, se conecta configurando este endereço como bootstrap.servers (ou equivalente) — é a partir dele que o cliente descobre o restante do cluster.

> Este endereço é único para todo o projeto. Não criar um broker Kafka paralelo no ambiente local do componente além do protótipo de testes do próprio squad — o objetivo é que todos os 15 squads integrem no mesmo barramento real.

2. Cada componente usa o cliente Kafka da sua linguagem

Os componentes do sistema estão sendo desenvolvidos em linguagens diferentes conforme a squad e a natureza do trabalho (inferência, serviço, front, etc.). Isso é esperado e não é um problema: Kafka não exige que produtor e consumidor estejam na mesma linguagem ou no mesmo processo. O protocolo de comunicação com o broker é padronizado, e cada linguagem tem sua própria biblioteca cliente que implementa esse protocolo.

Ou seja: um componente escrito em Python publica normalmente em um tópico, e um componente escrito em C++ consome esse mesmo tópico sem qualquer tradução ou adaptação — o envelope da mensagem (ver seção 4) é quem garante que os dois lados se entendam, não a linguagem de implementação.

2.1 Biblioteca recomendada por linguagem

Linguagem	Biblioteca cliente recomendada	Observação
Python	confluent-kafka (confluent-kafka-python)	Wrapper sobre librdkafka; preferir a aiokafka apenas se o componente já for assíncrono (asyncio) ponta a ponta.
C++ / C	librdkafka	Biblioteca nativa usada por quase todos os outros clientes por baixo; indicada para componentes de inferência com restrição de latência na Jetson.
Java / Kotlin	org.apache.kafka:kafka-clients (cliente oficial Apache Kafka)	Cliente de referência do projeto Kafka; mesma JVM pode usar Spring Kafka se o componente já usa Spring.
Node.js / TypeScript	kafkajs	Boa opção para os componentes de camada 4 que já expõem front em Node.
Go	confluent-kafka-go ou segmentio/kafka-go	confluent-kafka-go embute librdkafka (requer CGO); segmentio/kafka-go é pure Go, mais simples de compilar na Jetson.
C# / .NET	Confluent.Kafka	Cliente oficial mantido pela Confluent para .NET.

Se o componente for escrito em uma linguagem não listada acima, verificar primeiro se existe um cliente oficial ou amplamente adotado antes de usar uma alternativa — praticamente toda linguagem de uso corrente tem um cliente Kafka maduro disponível.

3. Tópicos e nomenclatura

Cada tipo de evento tem o seu próprio tópico. O nome segue o mesmo padrão hierárquico do campo schema do envelope de mensagem:

```
ods.<dominio>.<nome_do_evento>
```

- ods — prefixo fixo do projeto.
- <dominio> — a área funcional de onde o evento se origina (não o nome do componente, que pode mudar de dono ao longo do projeto).
- <nome_do_evento> — nomeado como fato já ocorrido (particípio passado ou substantivo de resultado), nunca como uma ordem ou pergunta.

Nomes de domínio e de evento em snake_case, minúsculo, sem acentos. A versão do schema não entra no nome do tópico — ela viaja dentro da mensagem, no campo schema_version do envelope.

3.1 Domínios e tópicos definidos

Domínio	Exemplos de tópico	Componentes produtores
visao	`ods.visao.objeto_detectado` `ods.visao.identidade_atribuida`	I1, I2, I3
espaco	`ods.espaco.zona_ocupada` `ods.espaco.zona_desocupada`	S1
regra	`ods.regra.evento_consolidado`	S2
midia	`ods.midia.evidencia_registrada`	S4
metrica	`ods.metrica.agregacao_calculada`	S5
alerta	`ods.alerta.emitido` `ods.alerta.reconhecido`	S3
fala	`ods.fala.transcricao_produzida`	I5
gesto	`ods.gesto.reconhecido`	I4, S6
acesso	`ods.acesso.codigo_lido`	I6, A2
veiculo	`ods.veiculo.placa_lida`	I7
epi	`ods.epi.conformidade_avaliada`	I9
industria	`ods.industria.defeito_classificado`	I8

> Esta lista é provisória. Os tópicos acima são sugestões de domínio e nomenclatura para orientar os squads, não uma enumeração fechada. Se um componente precisar de um tipo de evento que não se encaixa em nenhum domínio listado, ou de um domínio novo, o squad pode especificar o tópico seguindo o mesmo padrão ods.<dominio>.<nome_do_evento> descrito acima.

Esta lista é viva: à medida que novos tipos de evento forem especificados por qualquer squad, o ideal é comunicar à squad TRA-1 (B1/B2) o nome adotado, para manter este documento atualizado e evitar que dois squads criem domínios diferentes para a mesma coisa.

4. Como usar o serviço

O procedimento abaixo é o mesmo em qualquer linguagem; muda apenas a sintaxe da biblioteca cliente. Os exemplos usam Python (confluent-kafka) e Java (kafka-clients) como referência — para as demais linguagens, a lógica é

idêntica: configurar o endereço do broker, serializar a mensagem em JSON seguindo o envelope comum, e publicar ou assinar o tópico desejado.

4.1 Publicar um evento (produtor)

Passos:

- Configurar o cliente com o endereço do broker (34.66.235.91:9093).
- Montar a mensagem seguindo o envelope comum definido por B1/B2 (message_id, message_type, schema, schema_version, producer, occurred_at, payload, etc.).
- Serializar o envelope como JSON.
- Publicar no tópico correspondente ao tipo de evento.

Exemplo em Python

```
# Biblioteca cliente Kafka para Python
from confluent_kafka import Producer
import json, uuid, datetime

# Conecta ao broker do projeto (endereço de bootstrap)
produtor = Producer({'bootstrap.servers': '34.66.235.91:9093'})

# Monta o envelope da mensagem: campos comuns a todo evento
evento = {
    "message_id": str(uuid.uuid4()),      # id único desta mensagem
    "message_type": "event",              # tipo: evento (fato já ocorrido)
    "schema": "ods.visao.objeto_detectado",  # tipo de evento (igual ao tópico)
    "schema_version": "1.0",              # versão do formato do payload
    "producer": "I1",                     # componente que está publicando
    "occurred_at": datetime.datetime.utcnow().isoformat() + "Z",  # quando o fato ocorreu
    # payload: conteúdo específico deste tipo de evento
    "payload": {"class": "person", "confidence": 0.94}
}

# Publica no tópico do evento; o Kafka só aceita bytes, então
# o envelope é convertido em JSON e depois em UTF-8
produtor.produce(
    topic="ods.visao.objeto_detectado",
    value=json.dumps(evento).encode("utf-8")
)

# O envio é assíncrono (fica em um buffer interno).
# flush() espera até que tudo tenha sido de fato entregue ao broker
produtor.flush()
```

Exemplo em Java

```
// Configurações do cliente, em formato chave/valor
Properties props = new Properties();

// Endereço do broker do projeto (endereço de bootstrap)
props.put("bootstrap.servers", "34.66.235.91:9093");

// Como converter chave e valor da mensagem em bytes.
// Aqui ambos são texto (o envelope JSON é enviado como String)
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

// Cria o produtor: chave String, valor String
KafkaProducer<String, String> produtor = new KafkaProducer<>(props);

// Envelope já serializado como JSON (reticências = demais campos do envelope)
String envelope = "{ \"message_id\": \"...\", \"schema\": \"ods.visao.objeto_detectado\", ... }";

// Publica no tópico do evento; o envio é assíncrono
produtor.send(new ProducerRecord<>("ods.visao.objeto_detectado", envelope));
```

```
// Garante que tudo que está no buffer foi entregue ao broker
produtor.flush();
```

4.2 Consumir um evento (consumidor)

Passos:

- Configurar o cliente com o endereço do broker e um group.id próprio do componente consumidor — cada componente deve ter seu próprio group.id, para que o reprocessamento ou a lentidão de um consumidor não afete o progresso de leitura de outro.
- Assinar (subscribe) o tópico ou os tópicos de interesse.
- Ler as mensagens, desserializar o JSON e validar contra o schema esperado antes de processar o payload.

Exemplo em Python

```
# Biblioteca cliente Kafka para Python
from confluent_kafka import Consumer
import json

consumidor = Consumer({
    'bootstrap.servers': '34.66.235.91:9093',  # endereço do broker
    'group.id': 'S1-zonas',   # identifica este componente; cada componente usa o seu
    'auto.offset.reset': 'earliest'  # sem posição salva, começa da mensagem mais antiga
})

# Escolhe quais tópicos este componente quer receber (pode ser mais de um)
consumidor.subscribe(['ods.visao.objeto_detectado'])

while True:
    # Espera até 1 segundo por uma mensagem; devolve None se nada chegou
    msg = consumidor.poll(1.0)

    # Nada chegou ou houve erro de leitura: volta e tenta de novo
    if msg is None or msg.error():
        continue

    # Converte os bytes recebidos de volta em um dicionário Python
    evento = json.loads(msg.value().decode('utf-8'))

    # processar evento['payload']
    # (valide o envelope antes de usar o conteúdo: ver seção 4.3)
```

Exemplo em Java

```
// Configurações do cliente, em formato chave/valor
Properties props = new Properties();

// Endereço do broker do projeto (endereço de bootstrap)
props.put("bootstrap.servers", "34.66.235.91:9093");

// Identifica este componente; cada componente consumidor usa o seu
props.put("group.id", "S1-zonas");

// Como converter os bytes recebidos de volta em texto
props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");
props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDeserializer");

KafkaConsumer<String, String> consumidor = new KafkaConsumer<>(props);

// Escolhe quais tópicos este componente quer receber (pode ser mais de um)
consumidor.subscribe(List.of("ods.visao.objeto_detectado"));

while (true) {
    // Espera até 1 segundo por mensagens; devolve lista vazia se nada chegou
    ConsumerRecords<String, String> records = consumidor.poll(Duration.ofMillis(1000));

    // Percorre cada mensagem recebida
    for (ConsumerRecord<String, String> record : records) {
```

```
        // processar record.value()  (é o envelope JSON em texto)
        // valide o envelope antes de usar o conteúdo: ver seção 4.3
    }
}
```

4.3 Validar o envelope recebido

Todo consumidor deve conferir se a mensagem recebida está de acordo com o que espera antes de usar o conteúdo. A validação tem três níveis: a mensagem é um JSON válido; o envelope tem os campos obrigatórios e o tipo de evento esperado; e o conteúdo (payload) tem a estrutura e os valores que o componente sabe processar. Se qualquer nível falhar, a mensagem é descartada e o motivo é registrado em log — nunca processada "na tentativa".

Exemplo em Python

```
import json, logging

# Campos que todo envelope precisa ter
CAMPOS_OBRIGATORIOS = [
    'message_id', 'message_type', 'schema', 'schema_version',
    'producer', 'occurred_at', 'payload'
]
# Tipo de evento que este componente sabe processar
SCHEMA_ESPERADO = 'ods.visao.objeto_detectado'
# Versão do formato aceita: 1.0, 1.1, ... mas não 2.0
VERSAO_SUPORTADA = '1.'

def validar_envelope(evento):
    # Devolve (True, None) se estiver tudo certo,
    # ou (False, "motivo") se a mensagem deve ser descartada

    # 1) O envelope tem todos os campos obrigatórios?
    for campo in CAMPOS_OBRIGATORIOS:
        if campo not in evento:
            return False, 'campo obrigatório ausente: ' + campo

    # 2) É o tipo de evento e a versão que este componente espera?
    if evento['schema'] != SCHEMA_ESPERADO:
        return False, 'schema inesperado: ' + str(evento['schema'])
    if not str(evento['schema_version']).startswith(VERSAO_SUPORTADA):
        return False, 'versão não suportada: ' + str(evento['schema_version'])

    # 3) O conteúdo (payload) está no formato que este componente entende?
    payload = evento['payload']
    if not isinstance(payload, dict):
        return False, 'payload não é um objeto'
    if 'class' not in payload:
        return False, 'payload sem o campo class'
    conf = payload.get('confidence')
    # a confiança precisa ser um número entre 0 e 1
    if not isinstance(conf, (int, float)) or not 0 <= conf <= 1:
        return False, 'confidence ausente ou fora do intervalo 0 a 1'

    return True, None


# Uso dentro do laço do consumidor (depois do consumidor.poll)
msg = consumidor.poll(1.0)
if msg is not None and not msg.error():
    try:
        # Primeiro: o texto recebido é um JSON válido?
        evento = json.loads(msg.value().decode('utf-8'))
    except (json.JSONDecodeError, UnicodeDecodeError):
        # Não era JSON: descarta e registra no log
        logging.warning('mensagem descartada: JSON inválido')
    else:
        valido, motivo = validar_envelope(evento)
        if valido:
            pass  # processar evento['payload']
        else:
```

```
            # Fora do formato esperado: descarta e registra o motivo
            logging.warning('mensagem descartada: %s', motivo)
```

Para eventos com muitos campos, em vez de escrever as verificações à mão é possível descrever o formato esperado como um JSON Schema e usar uma biblioteca de validação da linguagem (por exemplo, jsonschema em Python, ajv em Node.js ou networknt/json-schema-validator em Java). A lógica é a mesma: validar, descartar e registrar quando não estiver conforme.

4.4 Pontos de atenção ao usar o serviço

- Assumir que uma mensagem pode chegar duplicada: o consumidor deve ser capaz de processar o mesmo evento mais de uma vez sem gerar efeito colateral duplicado (idempotência no lado do consumidor).
- Não embutir vídeo, áudio, imagem ou arquivo grande no payload — apenas referências (stream_id, media_ref, etc.). O Kafka transporta mensagens pequenas.
- Validar o envelope recebido contra o esperado antes de processar (ver seção 4.3); uma mensagem fora do formato deve ser descartada e registrada.

4.5 Cuidado com o consumo de créditos do Google Cloud

> O serviço roda em uma instância paga no Google Cloud, compartilhada por todos os squads. Loops de publicação ou consulta sem controle, reconexões repetidas em sequência e sondagens (poll) em intervalo muito curto geram tráfego constante no broker e consomem créditos de forma desnecessária — inclusive fora do horário em que alguém está efetivamente testando. Isso pode esgotar o crédito disponível antes do fim do semestre e tirar o serviço do ar para todo mundo.

- Evitar laços de repetição sem condição de parada ao publicar ou consultar — principalmente em código de teste deixado rodando sem supervisão.
- Não deixar um consumidor de teste publicando ou lendo em polling agressivo (poll em loop apertado, sem pausa) por longos períodos parado sem necessidade.
- Encerrar o processo (produtor ou consumidor) ao final do teste, em vez de deixá-lo aberto indefinidamente em segundo plano.
- Evitar scripts de carga ou testes de estresse apontando para o serviço em nuvem sem combinar antes com a squad TRA-1, já que o impacto é sobre o crédito de todo o projeto, não apenas do squad que está testando.

Dúvidas sobre nomenclatura de tópico, criação de um tópico novo ou formato do envelope devem ser direcionadas à squad TRA-1 (componentes B1 e B2).